

Implementación de Consistencia y Prevención de Split-Brain

Resumen

Se implementaron las funcionalidades faltantes del plan original para prevenir split-brain activo y resolver conflictos determinísticamente tras reunificación de red.

1. Lease + Primary Epoch en DNS ✓

Archivos modificados:

- `app/dns_service/main.py`

Implementación:

- **Epoch global monótonico** (`primary_epoch`) que se incrementa en cada promoción de PRIMARY
- **Lease con duración configurable** (`LEASE_DURATION`, default 10s)
- **Emisión de lease** en:
 - `POST /server/register`: Asigna epoch y lease al nuevo PRIMARY
 - `POST /server/heartbeat`: Renueva lease del PRIMARY actual
 - `GET /server/resolve`: Emite epoch y lease al promover BACKUP
- **Respuesta extendida** en `APIServerResolveResponse`:
 - `primary_epoch`: Número de epoch actual
 - `lease_expires_at`: Timestamp ISO de expiración del lease

Ejemplo de flujo:

```
1. Storage_1 se registra → DNS asigna PRIMARY, epoch=1,
lease_expires=T+30s
2. Storage_1 envía heartbeat cada 10s → DNS renueva lease a T+30s
3. Storage_1 se desconecta → lease expira sin renovación
4. Storage_2 (backup) envía heartbeat → DNS detecta PRIMARY caído,
promueve Storage_2 con epoch=2
```

2. Fencing en Storage ✓

Archivos modificados:

- `app/storage/services/node_manager.py`
- `app/storage/api/endpoints.py`

NodeManager

- **Nuevas propiedades:**
 - `_primary_epoch`: Epoch recibido del DNS
 - `_lease_expires_at`: Timestamp de expiración del lease
 - `has_valid_lease()`: Valida si el lease sigue vigente (compara con `time.time()`)
- **Guardar epoch/lease** al registrarse y recibir heartbeat response

Endpoints

- **POST /upload**: Validación de fencing antes de aceptar escrituras

```
if not node_mgr.is_primary:
    raise HTTPException(503, {"error": "not_primary", ...})

if not node_mgr.has_valid_lease():
    raise HTTPException(409, {"error": "lease_expired", "epoch":
...})
```

- **Cálculo de hash SHA256** del contenido durante upload
- **Guardar metadata extendida:** `content_hash`, `origin_node`, `write_epoch` en DB

Resultado: Un nodo aislado (ex-PRIMARY) **rechaza escrituras** tan pronto su lease expira, evitando divergencia durante partición.

3. Processor Tolerante a Lease Expired ✓

Archivos modificados:

- `app/processor/services/storage_client.py`

Implementación:

- **Detección de errores de fencing** en `_make_request()`:
 - Códigos `409 Conflict` o `503 Service Unavailable`
 - Con payload `{"error": "lease_expired"}` o `{"error": "not_primary"}`
- **Retry automático:**
 1. Al detectar fencing error → invalida cache DNS
 2. Re-resuelve PRIMARY actual desde DNS
 3. Reintenta request con el PRIMARY válido

Ejemplo de flujo:

1. Processor envía POST /upload a Storage_1 (ex-PRIMARY aislado)
2. Storage_1 responde 409 {"error": "lease_expired"}
3. Processor invalida cache DNS y consulta GET /server/resolve
4. DNS responde Storage_2 (nuevo PRIMARY con epoch=2)
5. Processor reintenta POST /upload a Storage_2 → ✓ éxito

4. Versionado y Hash ✓

Archivos modificados:

- `app/storage/db/schema.sql`
- `app/storage/db/crud.py`

Schema

Nuevas columnas en tabla `files`:

```
content_hash TEXT DEFAULT NULL,      -- SHA256 del contenido
version INTEGER DEFAULT 1,          -- Contador monótonico por
file_path
origin_node TEXT DEFAULT NULL,      -- Nodo que escribió esta versión
write_epoch INTEGER DEFAULT NULL    -- Epoch del PRIMARY al escribir
```

CRUD

- `upsert_file()`: Acepta parámetros adicionales y auto-incrementa `version` si no se provee
- `get_file()`, `list_files()`, `search_files()`: Incluyen nuevos campos en SELECT

5. Cálculo de Hash en Upload y Sync ✓

Archivos modificados:

- `app/storage/api/endpoints.py`
- `app/storage/services/sync_service.py`

Upload

- Leer contenido completo antes de guardar
- Calcular `hashlib.sha256(content).hexdigest()`
- Guardar hash en DB junto con `origin_node` y `write_epoch`

Sync (Pull de PRIMARY)

- `GET /internal/files` devuelve `content_hash` en lista de archivos
- Backup calcula hash local y compara con remoto
- Detecta divergencia incluso con mismo `last_modified`

Replicación (Push al PRIMARY)

- `POST /internal/replicate-file` acepta `content_hash`, `origin_node`, `write_epoch`
 - Verifica hash al recibir si se provee
 - Guarda metadata completa en DB
-

6. Resolución de Conflictos con Rename ✓

Archivos modificados:

- `app/storage/services/sync_service.py`

Implementación:

```
# Detectar conflicto: mismo mtime pero hash distinto
if (remote_hash and local_hash and
    remote_hash != local_hash and
    abs(remote_mtime - local_mtime) <= tolerance):

    # Preservar local como copia de conflicto
    conflict_name = f"{stem}.conflict.{local_hash[:8]}{suffix}"
    shutil.copy2(local_path, conflict_path)

    # Descargar versión del PRIMARY (converger a PRIMARY)
    action = "download"
```

Resultado: Ante conflictos, se preservan **ambas versiones**:

- Versión del PRIMARY → mantiene nombre original
- Versión del BACKUP → renombrada a `*.conflict.<hash_corto>`

Ejemplo:

```
# Antes de reunificación
PRIMARY: /files/doc.txt (hash=abc12345, mtime=10:00:00)
BACKUP:  /files/doc.txt (hash=def67890, mtime=10:00:00)

# Después de sync
BACKUP:  /files/doc.txt (hash=abc12345, del PRIMARY)
BACKUP:  /files/doc.conflict.def67890.txt (versión local preservada)
```

PROF

Resumen de Protecciones Implementadas

Escenario	Mecanismo	Resultado
Partición de red activa	Lease expiration + fencing	Ex-PRIMARY rechaza escrituras → consistencia
Processor envía a nodo incorrecto	Retry con 409/503	Re-resuelve y reintenta con PRIMARY real
Mismo archivo modificado en ambos lados	Hash + LWW + rename	Converge a PRIMARY, preserva BACKUP como <code>.conflict</code>

Escenario	Mecanismo	Resultado
Reloj desincronizado entre nodos	Hash detecta divergencia	No depende solo de mtime para conflictos
Múltiples primaries tras merge	Epoch monotónico	DNS arbitra con epoch más alto

Variables de Entorno Nuevas

DNS

- `LEASE_DURATION`: Duración del lease en segundos (default: 30)
- `HEARTBEAT_INTERVAL`: Intervalo de heartbeat en segundos (default: 10 en storage)
- `HEALTH_CHECK_INTERVAL`: Intervalo de health check en DNS (default: 10)
- `API_SERVER_TIMEOUT`: Timeout para considerar storage caído (default: 25)

Storage

(Usa valores del DNS vía heartbeat response)

Consideraciones de Despliegue

1. **Sincronización de relojes**: Aunque hash mitiga problemas, NTP/chrony mejora LWW
2. **Lease duration vs heartbeat interval**: Configurado como `LEASE_DURATION = 3 * HEARTBEAT_INTERVAL` (30s / 10s) para tolerar hasta 2 heartbeats perdidos
3. **Migración de schema**: Los nodos existentes necesitarán recrear DB o ejecutar `ALTER TABLE` manualmente
4. **Performance**: Cálculo de hash añade overhead en uploads; tolerable para archivos <10MB

Pruebas Recomendadas

1. Split-brain prevention:

- Desconectar storage PRIMARY de red overlay
- Intentar upload desde client → debe fallar tras lease expiry
- Verificar que BACKUP es promovido y acepta nuevos uploads

2. Conflict resolution:

- Durante partición, subir archivo con mismo nombre en ambos lados
- Reconectar red
- Verificar que aparece `*.conflict.*` en el backup

3. Lease renewal:

- Observar logs de PRIMARY durante heartbeats
- Verificar que `lease_expires_at` se renueva cada ciclo

Estado Final

✓ Todas las funcionalidades del plan implementadas:

- ☒ Paso 1: Auditar código (completado previamente)
- ☒ Paso 2: Lease + primary_epoch en DNS
- ☒ Paso 3: Fencing en Storage
- ☒ Paso 4: Processor tolerante a errores de fencing
- ☒ Paso 5: Versionado y hash en schema/CRUD
- ☒ Paso 6: Resolución de conflictos con rename

El sistema ahora previene split-brain durante particiones activas y converge determinísticamente tras reunificación preservando datos.